

Parallel Programming

Embarrassingly Parallel Computations (2)

Dr. Marwa Dahdouh

Mandelbrot set

Mandelbrot set is another example of processing a bit-mapped image. The image must be computed, and this involves significant computation. A Mandelbrot set is a set of points in a complex plane that are quasi-stable when computed by iterating a function:

$$z_{k+1} = z_k^2 + c$$

where z_{k+1} is the $(k+1)$ th iteration of the complex number $z = a + bi$ (where i

Mandelbrot set (cont.)

The initial value for z is zero.

The iterations are continued until the magnitude of z is greater than 2 or the number of iterations reaches some arbitrary limit. The magnitude of z is the length of the vector given by:

$$z_{length} = \sqrt{a^2 + b^2}$$

Computing the complex function $z_{k+1} = z_k^2 + c$, is simplified by recognizing that

$$z^2 = a^2 + 2abi + bi^2 = a^2 - b^2 + 2abi$$

real part that is $a^2 - b^2$ and an imaginary part is $2ab$.

Mandelbrot set (cont.)

if z_{real} is the real part of z , and z_{imag} is the imaginary part of z , the next iteration values can be produced by computing:

$$z_{real} = z_{real}^2 - z_{imag}^2 + c_{real}$$

$$z_{imag} = 2z_{real}z_{imag} + c_{imag}$$

where c_{real} is the real part of c , and c_{imag} is the imaginary part of c .

Mandelbrot set: Sequential Code

For coding, a structure can be used holding both the real and imaginary parts of z :

```
structure complex {  
    float real;  
    float imag;  
};
```

A routine for computing the value of one point and returning the number of iterations could be of the form (in the next slide):

Mandelbrot set: Sequential Code (cont.)

```
int cal_pixel(complex c)
{
    int count, max_iter;
    complex z;
    float temp, lengthsq;
    max_iter = 256;
    z.real = 0;
    z.imag = 0;
    count = 0;                                     /* number of iterations */
    do {
        temp = z.real * z.real - z.imag * z.imag + c.real;
        z.imag = 2 * z.real * z.imag + c.imag;
        z.real = temp;
        lengthsq = z.real * z.real + z.imag * z.imag;
        count++;
    } while ((lengthsq < 4.0) && (count < max_iter));
    return count;
}
```

Mandelbrot set: Sequential Code (cont.)

The square of the length, `lengthsq`, is compared against 4, rather than the length against 2, to avoid a square root operation.

Given the termination conditions, all the Mandelbrot points must be within a circle with its center at the origin and of radius 2.

Mandelbrot set: Sequential Code (cont.)

The code for computing and displaying the points requires some scaling of the coordinate system to match the coordinate system of the display area.

The actual viewing area will usually be a rectangular window of any size and sited anywhere of interest in the complex plane.

The resolution is expanded at will to obtain fascinating images.

Mandelbrot set: Sequential Code (cont.)

Suppose the display height is **disp_height**, and the display width is **disp_width**, and the point in this display area is **(x, y)**.

If this window is to display the complex plane with minimum values of (real_min, imag_min) and maximum values of (real_max, imag_max), each(x, y) point needs to be scaled by the factors

```
c.real = real_min + x * (real_max - real_min) / disp_width;  
c.imag = imag_min + y * (imag_max - imag_min) / disp_height;
```

Mandelbrot set: Sequential Code (cont.)

To obtain the actual complex plane coordinates. For computational efficiency, let:

```
scale_real = (real_max - real_min)/disp_width;  
scale_imag = (imag_max - imag_min)/disp_height;
```

Mandelbrot set: Sequential Code (cont.)

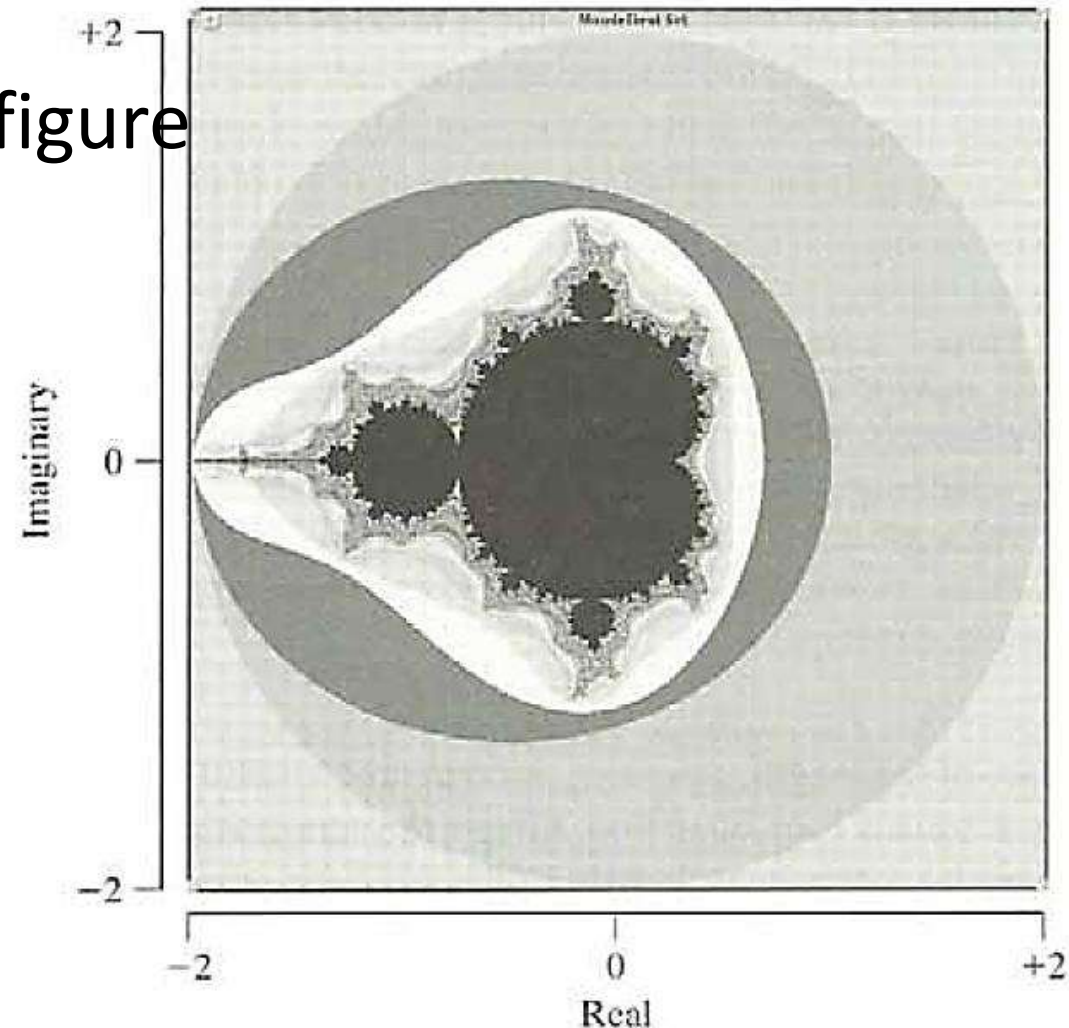
Including scaling, the code could be of the form:

```
for (x = 0; x < disp_width; x++)      /* screen coordinates x and y */
    for (y = 0; y < disp_height; y++) {
        c.real = real_min + ((float) x * scale_real);
        c.imag = imag_min + ((float) y * scale_imag);
        color = cal_pixel(c);
        display(x, y, color);
    }
```

where `display()` is a routine suitably written to display the pixel (x, y) at the computed color.

Mandelbrot set: The Result

Typical results are shown in this figure



Note

The **Mandelbrot** set is a widely used test in parallel computer systems and sequential computers because it is computationally intensive.

It can take several minutes to compute the complete image, depending upon the speed of the computer and the required resolution of the image.

Also, it has interesting graphical results.

Parallelizing the Mandelbrot Set Computation

The Mandelbrot set is particularly convenient to parallelize for a message-passing system because each pixel can be computed without any information about the surrounding pixels.

The computation of each pixel itself is less amenable to parallelization.

We will consider both **static assignment** and **dynamic assignment** when the areas computed by a process vary.

Mandelbrot Set : Static Task Assignment

Grouping by square/rectangular regions or by columns/rows is suitable.

Each process needs to execute the procedure, `cal_pixel()`, after being given the coordinates of the pixels to compute.

Suppose the display area is 640 x 480 and we were to compute 10 rows in a process (i.e., a grouping of 10 x 640 pixels and 48 processes).

The pseudocode might look like the following (next slides).

Mandelbrot Set : Static Task Assignment (cont.)

Master

```
for (i = 0, row = 0; i < 48; i++, row = row + 10) /* for each process */
    send(&row, Pi);                                /* send row no. */
for (i = 0; i < (480 * 640); i++) {                /* from processes, any order */
    recv(&X,&Y,&color, PANY);                        /* receive coordinates/colors */
    display(x, y,color);                             /* display pixel on screen */
}
```

Mandelbrot Set : Static Task Assignment (cont.)

Slave (process i)

```
recv(&row, P_master);          /* receive row no. */
for (x = 0; x < disp_width; x++) /* screen coordinates x and y */
    for (y = row; y < (row + 10); y++) {
        c.real = min_real + ((float) x * scale_real);
        c.imag = min_imag + ((float) y * scale_imag);
        color = cal_pixel(c);
        send(&x, &y, &color, P_master); /* send coords, color to master */
    }
```

Mandelbrot Set : Dynamic Task Assignment

Dynamic Task Assignment - Work Pool/Processor Farms:

The Mandelbrot set requires significant iterative computation for each pixel.

The number of iterations will generally be different for each pixel.

In addition, the computers may be of different types or operate at different speeds.

Thus, some processors may complete their assignments before others.

Mandelbrot Set : Dynamic Task Assignment (cont.)

Ideally, We want all processors to complete together. achieving a system efficiency 100 percent, which can addressed using **load balancing**.

Ideally, each processor needs to be given sufficient work to keep it busy for the duration of the overall computation.

Mandelbrot Set : Dynamic Task Assignment (cont.)

Regions of different sizes could be assigned to different processors, but this would not be very satisfactory for two reasons:

- We may not know a priori each processor's computational speed,
- and we would have to know the exact time it takes for each processor to compute each pixel. The latter depends upon the number of iterations, which differs from one pixel to the next.

Mandelbrot Set : Dynamic Task Assignment (cont.)

Dynamic load balancing can be achieved using a work-pool approach, in which individual processors are supplied with work when they become idle.

Sometimes the term processor farm is used to describe this approach, especially when all the processors are of the same type.

The work pool holds a collection, or pool, of tasks to be performed.

Mandelbrot Set : Dynamic Task Assignment (cont.)

In our problem, the set of pixels (or, more accurately, their coordinates) forms the tasks.

The number of tasks is fixed in that the number of pixels to compute is known prior to the start of the computation.

Individual processors request a pair of pixel coordinates from the work pool.

Mandelbrot Set : Dynamic Task Assignment (cont.)

When a processor has computed the color for the pixel, it returns the color and requests a further pair of pixel coordinates from the work pool.

When all the pixel coordinates have been taken, we then need to wait for all the processors to complete their tasks and report in for more pixel coordinates.

Mandelbrot Set : Dynamic Task Assignment (cont.)

Sending pairs of coordinates of individual pixels will result in excessive communication.

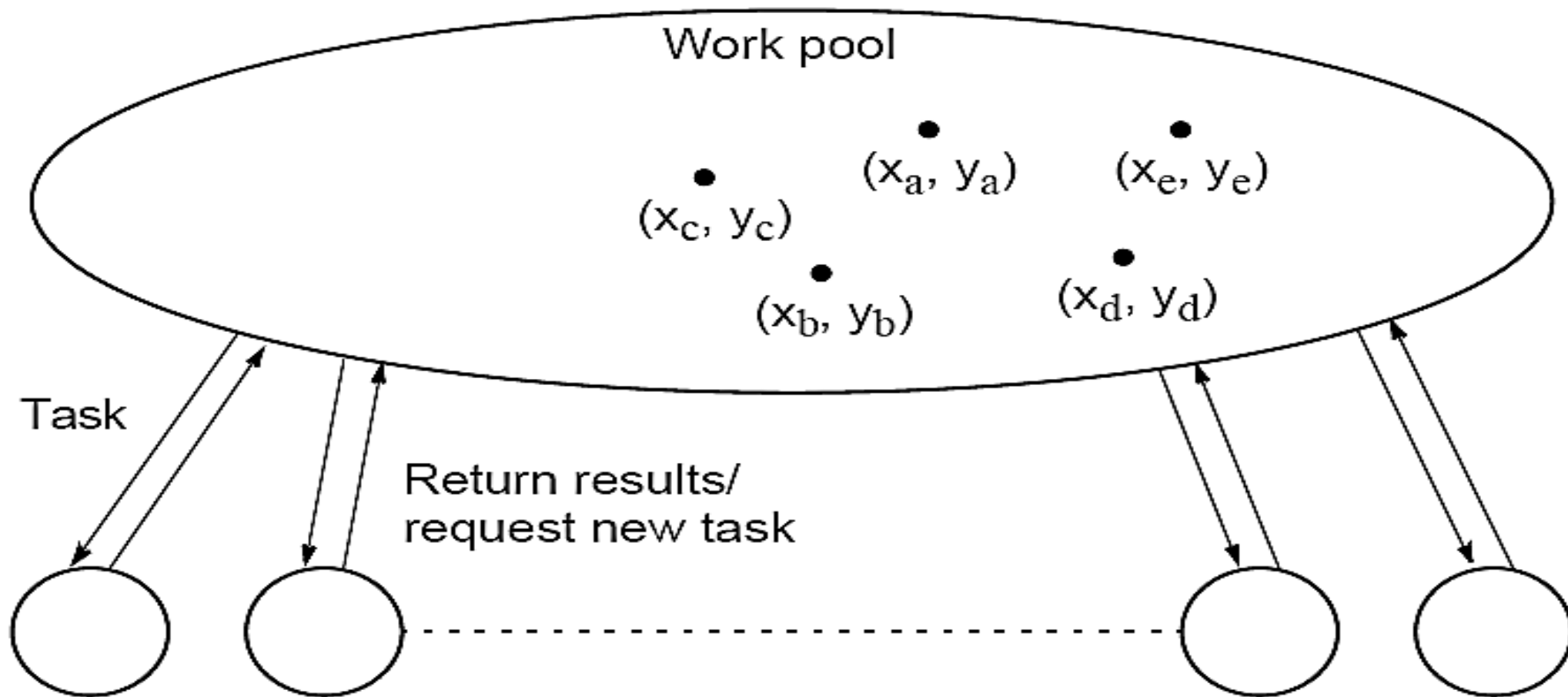
Normally, rather than a single coordinate comprising a task, a group of coordinates representing several pixels will form the task to be taken from the work pool in order to reduce the communication overhead.

At the start, the slaves are told the size of the group of pixels (assumed to be a fixed size). Then only the first pair of coordinates of the group need be sent to the slaves as a task.

This approach reduces the communication to an acceptable level.

The overall arrangement is depicted in the next figure.

Mandelbrot Set : Dynamic Task Assignment (cont.)



Mandelbrot Set : Dynamic Task Assignment (cont.)

Suppose the number of processes is given by **num_proc**, and processes compute one row at a time.

In this case, the work pool holds row numbers rather than individual pixel coordinates.

Coding for a work pool could be of the form (next slides).

Mandelbrot Set : Dynamic Task Assignment (cont.)

Master

```
count = 0;                                /* counter for termination*/
row = 0;                                  /* row being sent */
for (k = 0; k < num_proc; k++) {          /* assuming num_proc < disp_height */
    send(&row, Pk, data_tag);            /* send initial row to process */
    count++;                             /* count rows sent */
    row++;                               /* next row */
}

do {
    recv (&slave, &r, color, PANY, result_tag);
    count--;                             /* reduce count as rows received */
    if (row < disp_height) {
        send (&row, Pslave, data_tag);    /* send next row */
        row++;                             /* next row */
        count++;
    } else
        send (&row, Pslave, terminator_tag); /* terminate */
    display (r, color);                   /* display row */
} while (count > 0);
```

Mandelbrot Set : Dynamic Task Assignment (cont.)

Slave

```
recv(&y, P_master, ANYTAG, source_tag);    /* receive 1st row to compute */
while (source_tag == data_tag) {
    c.imag = imag_min + ((float) y * scale_imag);
    for (x = 0; x < disp_width; x++) {      /* compute row colors */
        c.real = real_min + ((float) x * scale_real);
        color[x] = cal_pixel(c);
    }
    send(&i, &y, color, P_master, result_tag); /* row colors to master */
    recv(&y, P_master, ANYTAG, source_tag);    /* receive next row */
}
```

Mandelbrot Set : Dynamic Task Assignment (cont.)

In this code, each slave is first given one row to compute and, from then on, gets another row when it returns a result, until there are no more rows to compute.

The master sends a terminator message when all the rows have been taken.

To differentiate between different messages, tags are used with the message **data_tag** for rows being sent to the slaves, **terminator_tag** for the terminator message, and **result_tag** for the computed results from the slaves. Here we simply show a **source_tag** parameter.